"FINALS"

"ASSIGNMENT # 02"

1) Array # Q1 :

1) `int n = sizeof(array) / sizeof(array[0]);`

• For finding the size of the array

2)
```
If(n==0)
{
   return 0;
}
```

• When there is no element in the array.

3) `int j=0;`

• We will use this for tracking unique element of the array while comparing it with i.

4)
```
for(int i=0; i<n; i++)
{
  if(ary[i] != ary[j])
  {
     j++;
     ary[j] = ary[i];
  }
}
```

- We will run this loop until we reach arr[n] which is the last element of the array.

- If arr[i] != arr[j] we will increment j and store the value into arr[j].

- In this way every unique element will store in the arr[j].

3) int newsize = j + 1;

- We will create a new variable for the updated arr so assign it a value where the unique elements are stored.

(Dry Run)

- Suppose this is the array:

{1, 1, 2, 2, 3}.

sizeof(array) = 24
sizeof(arr[0]) = 4

So . n = 24/4 = 6
no of elements in arr = 6

Logic Loop:

i=0; j=0;

i<n ∴ 0<5 ✓

if(ary[i]!=ary[j]){-} ∴ 1!=1 X

i++;

i=1; j=0:

i<n ∴ 1<5 ✓

if(ary[i]!=ary[j]){-} ∴ 1!=1 X

i++;

i=2; j=0:

i<n ∴ 2<5 ✓

if(ary[i]!=ary[j]) ∴ 2!=1 ✓

{

j++; ∴ Now ary becomes

ary[j]=ary[i]; } {1,2,2,2,3}

i++

i=3; j=1;

i<n ∴ 3<5 ✓

if(ary[i]!=ary[j]) ∴ 2!=2 X

{ ∴ N{Re

j++;

ary[i] ←;

}

i=4; j=1;

i<n ∴ 4<5 ✓

if(ary[i]!=ary[j]) ∴ 3!=2 ✓

{ j++; ∴ j=2

ary[j]=ary[i]; } ary[2]=3

i+1 So: {1,2,3,2,3}

$i=5, j=2$: $\therefore 5<5$ X
$i<n$

So, loops end

Now,

int newsize = j+1; $\therefore 2+1=3$

This is the updated size of the array where we have unique elements.

X=====X=====X=====X

(Array_Q#02)

1) int n = sizeof(ary)/sizeof(ary[0]);

For finding no of elements in the array.

2) if(n==0){ return 0;}

When array is empty.

3) Sorting:
We will sort the ary first so we can easily find largest consecutive sequence.

(code -> later -> while -> Dry run)

4) int longest = 1;
int current = 1;

- We will use this longest to store the value of longest current.

- And will use current to check how long the consecutive numbers are.

5)
```
for(int i=0; i<n; i++)
{
    if(arr[i]+1 == arr[i+1])
    {
        current++;
    }
    else if(arr[i] != arr[i+1])
    {
        current=1;
    }

    if(current>longest)
    {
        longest=current;
    }
}
```

- We will run this code until it reaches i<n or i=n-1.

- if (arr[i]+1 == arr[i+1]) means if(0+1 == 1) update current (+1).

- else if(arr[i] != arr[i+1]) means when if condition fails means no consecutiv numbers then update the current again to 1.

- if(current > longest) moves the val of current to longest to preserve the longest consecutive arr elements.

(Dry Run)

Suppose this is the array:

{7, 2, 1, 3, 9, 11}

$\boxed{n = 6}$

<u>Sorted loops</u>

- i = 0, j = 0 :

~~i<n~~	∴ 0<6
j<n	∴ 0<6 ✓
if(arr[j] > arr[j+1]) {	∴ 7>2 ✓
int temp = arr[j];	temp = 7
arr[j] = arr[j+1];	7 → 2
arr[j+1] = ~~arr~~ temp;	2 → temp(7)
}	

arr becomes : {2, 7, 1, 3, 9, 11}

- i = 0 ; j = 1 ;

j<n	∴ 1<6

```
if(ary[j] > ary[j+1])                    7>1 ✓
{
    int temp = ary[j];                   temp = 7
    ary[j] = ary[j+1];                   7 -> 1
    ary[j+1] = temp;                     1 -> temp(7)
}
```

and so on, until the ary sorts.
sorted ary is {1, 2, 3, 7, 9, 11}

Logic loop:

•) loop until it reaches i<n || i=n-1;

```
For i=0;
        i<n                              0<6 ✓
if(ary[0]+1 == ary[0+1])                 1+1 = 2 ✓
{                                        curr = 2;
   current++;
}
else if(—) {—}                           X

if(current > longest)                    ✓
{
   longest = current;                    lon = 2;
}

For i=1;
        i<n                              1<6 ✓
if(ary[1]+1 == ary[1+1])                 2+1 = 3 ✓
{
   current++;                            curr = 3
}
else if(~){~}                            X
```

```
if (current > longest)                    ✓
{
    longest = current;                    lon = 3;
}
```

For i = 2 ;

```
        i < n                             2 < 6 ✓
    if (arr [2] + 1 == arr [2+1])         3+1 ≠ 7 ✗
    else if (arr[i] != arr [i+1])         3 ≠ 7 ✓
    {
        current = 1;                      curr = 1.
    }
    if (curr > lon) { — }                 1 > 3 ✗
              :
          and so on
```

For the supposed arr we first sorted and for that the longest consecutive sequence is 6.

X ═══════ X ═══════════════════ X ═══

(Array – Q # 03)

1) Size of array.

2) Sorting

3) IF array is empty

4) bool found=false;

• to prompt weather we found triplets or not

```
5) for(int i=0; i<n-2; i++)    // ~~suit~~
{
    if (i>0 && ary[i]==ary[i-1])
    {
        continue;
    }

    for(int j=~~0~~i+1; j<n-1; j++)
    {
        if(j>i+1 && ary[j]==ary[j-1])
        {
            continue;
        }

    ~~for~~ for(int k=~~j~~j+1; k<n; k++)
        {
            if(k>j+1 && ary[k]==ary[k-1])
            {
                continue;
            }

            if(ary[i]+ary[j]+ary[j]==0)
            {
                cout<<ary[i]<<"+"<<ary[j]<<"+"<<
                ary[k]<<"=0";
                found=true;
            }
} } }
```

•) We used 3 for loop for every 3 numbers in the array.

like this is the array:

```
[0, 1, 2, 3,
 ↓  ↓  ↓
 i  j  k
```

then

```
(0, 1, 2, 3, 4)
 ↓  ↓     ↓
 i  j     k
```

then

```
(0, 1, 2, 3, 4)
 ↓           ↓
 i           k
```

then

```
(0, 1, 2, 3, 4)
 ↓     ↓  ↓
 i     j  k
```

then

```
(0, 1, 2, 3, 4)
 ↓        ↓  ↓
 i        j  k
```

and so on until

```
(0, 1, 2, 3, 4)
       ↓  ↓  ↓
       i  j  k
```

and when arr[i] + arr[j] + arr[k] = 0 it will prompt it & make found = true.

(DRY RUN)

Logic_Loop:

Suppose this is the array

(-1, 0, 1, 2, -2)

after sorting: (-2, -1, 0, 1, 2)
0 1 2 3 4

n = 5

NOW:

For i=0, j=1, k=2:

i	j	k			
0	1	2	(-2,-1,0)	X	
0	1	3	(-2,-1,1)	X	
0	1	4	(-2,-1,2)	X	
		loop ends	5<5	X	j++;
0	2	3	(-2,0,1)	X	
0	2	4	(-2,0,2)	✓	Prompt
		loop end	5<5	X	j++;
0	3	4	(-2,1,2)	X	
		loop end	5<5	X	j++
	loop end		4<4	X	i++
1	2	3	(-1,0,1)	✓	prompt
1	2	4	(-1,0,2)	X	
		loop end	5<5	X	j++;

and so on

(Array _ Q # 04)

1) Size of Array

2) if array is empty

3)
```
for (int i=0; i<n-1; i++)
{
    for(j=0; j<n-1; j++)
    {
        if(ary[j]==0)
        {
            int temp= ary[j];
            ary[j] = ary[j+1];
            ary[j+1] =temp;
        }
    }
}
```

- Run the code until i<n-1 or i=n-1 and for compairing runs nested loop to compare the index.

- if 0 is found move it to the next index. (swap)

(DRY_RUN)

Ex Q.

Suppose the ary is = {0,1,0,0,2,3}

n=8

For i=0, j=0:

arr[0]==0 ✓

swap   0 ↔ 1

{1,0,0,0,2,3}

For i=0, j=1:

arr[1]==0; ✓

swap   0 ↔ 0

{1,0,0,0,2,3}

i=0, j=2:

arr[2]==0;   0 ↔ 0

swap   {1,0,0,0,2,3}

i=0, j=3;

arr[3]==0 ✓

swap(arr[j], arr[j+1])   0 ↔ 2

{1,0,0,2,0,3}

i=0, j=4;

arr[4]==0 ✓

swap   0 ↔ 3

{1,0,0,2,3,0}

j++;   X

j=5, 5<6-1

exit this loop

i++

i=1, j=0;

⋮ and so on

at last arr will become

{1,2,3,0,0,0}

(Array_Q#05)

1) Size of the array

2) if array is empty

3) int num = arr[0];

we will initially assign the first element as maxcount number.

4) int maxcount = 0;

creating it & initially assigning it 0.

5)
```
for(int i=0; i<n; i++)
{
    int count=0;

    for(int j=0; j<n; j++)
    {
        if(arr[i] == arr[j])
        {
            count++;
        }

        if(count>maxcount)
        {
            maxcount = count;
            num = arr[i];
        }
}
```

·) Starting nested loops to compare index i with index j.

·) if (arr[i] == arr[j]) : increase count when 2 same values found / when value same as arr[i] found.

·) if (count > maxcount) : if count is greter than maxcount then assign value of count to maxcount. also store that arr[i] value into num.

·) int count = 0 : when all values of nested loop compared reset count to 0.

(DRY_RUN)

Suppose this is arr = {1,1,1,2,2}
n = 5

i	j	count	mcount	num	
0	0	0	0	1	} Initially
0	0	1	1	1	
0	1	2	2	1	
0	2	3	3	1	
0	3	3	3	1	
0	4	3	3	1	
1	0	0	3	1	
1	0	1	3	1	

So on

(Array Q # 06)

1) Size of arr

2) int k=2;
   finding a sum of 2 in arr

3) int count=0;
   Initially set to 0

4)
```
for(int i=0; i<n; i++)
{
    int sum=0;
    for(int j=i; j<n; j++)
    {
        sum+=arr[j];
        if(sum==k)
        {
            count++;
        }
    }
}
```

•) Running nested loop &p j from i so can compare ~~it to next~~ every next 2 element of arr like:

{1,1,2,1}

{1,1,2,1}

{1,1,2,1}

- They can point to same index but j can not < i.
- if (sum == k) : when find sum = k then increment count.
- sum += arr[j] : if we get sum by more than 2 index that's adding why.

(DRY _ RUN)

Suppose arr = {1, 1, 2, 1}
n = 4
k = 2

i	j	sum	count
		0	0
~~0~~-	-		
		0	0
now 0	~~0~~-	1	0
0	0	2 ✓	1
0	1	4	1
0	2	5	1
0	3		
	4<4 x exit, i++		
1	1 (j=i)	~~0~~1	1
		3	1
1	2	4	1
1	3		
	~~4~~<4 x exit, i++		
2	2	2 ✓	2
	3		
	4<4 x exit i++		

so on

(Array — Q # 07)

1) Size of ary

2) int K=7;
num/sum we want to find.

3) int maxlength=0;
initially set to 0

4)
```
for(int i=0; i<n; i++)
{
    int sum=0;

    for(int j=i; j<n; j++)
    {
        sum+=ary[j];

        if(sum<=k)
        {
            int length=j-i+1;

            if(length>maxlength)
            {
                maxLength=length;
            }
        }
    }
}
```

•) Running nested loop until end where j≥i,

•) int sum=0: always set sum=0 for

incrementing outerloop:(i).

- if (sum <= k) : if sum <= k, calculate the length of array -> length = j-i+1 &
- if length > maxlength, update ml to l.
- sum += ary[j] -> add every iterated index wrt j to sum.

(DRY_RUN)

let ary = {1, 2, 3, ~~4~~, -2, -1}, n = 5, k = 2

i	j	sum	l	ml
		0	-	0
0	-	1	1	1
0	0	3	2	1
0	1	6	3	1
0	2	4	4	1
0	3	3	5	1
0	4			
	5<5 x exit i++			
1	1 (j=i)	2	1	1
1	2	5	2	1
1	3	3	3	1
1	4	2 ✓	4	4
	5<5, exit i++			
2	2 (i=j)			

so on